Course Registration Smart Contract

A Solidity smart contract that lets students register for and withdraw from courses, with automated tests and a web DApp.

Student	Joel
Department	Computer Science and Business Systems (CSBS), Final Year
Institution	KIT-Kalaignarkarunanidhi Institute of Technology, Coimbatore
Register No.	_____
Technology	Solidity 0.8.24, Remix IDE, ethers.js v6, Ganache, HTML/JavaScript DApp
Source code	https://github.com//course-registration-smart-contract

Deliverables: Solidity source code | Remix IDE screenshots | Test cases and results | Deployment/transaction details | GitHub/source-code link | DApp source code

Contents

1. Aim and problem statement

2. Design of the smart contract

3. Solidity source code (Outcome 1)

4. Remix IDE screenshots (Outcome 2)

5. Test cases and results (Outcome 3)

6. Deployment and transaction details (Outcome 4)

7. GitHub / source-code link (Outcome 5)

8. DApp source code (Outcome 6)

9. Conclusion

1. Aim and problem statement

Traditional course registration depends on a central administrator or portal. Records can be altered, are not transparent, and seat counts can be disputed. The aim of this project is to build a smart contract on Ethereum that handles registration and withdrawal in a transparent, tamper-proof way, where the rules (seat limits, one registration per course, maximum course load) are enforced by code and every action is recorded as an on-chain event.

Objectives

- Allow the admin to create courses with a fixed seat capacity and to activate or deactivate them.

- Allow any student (identified by wallet address) to register for an active course while seats are available.

- Allow students to withdraw from a course, which frees the seat for others.

- Enforce rules on-chain: no duplicate registration, no registration in full or inactive courses, a limit of 5 courses per student, and an admin-controlled registration window.

- Emit events for every state change so a front-end can track activity.

- Verify behaviour with an automated test suite and provide a simple web DApp.

2. Design of the smart contract

The contract **CourseRegistration** stores courses in a mapping keyed by course id. Each course has a name, capacity, enrolled count and an active flag. Two helper mappings keep the list of courses per student and the list of students per course, so both can be queried directly. Removal from these arrays uses the swap-and-pop technique, which keeps gas cost low.

Roles

Role	Who	Permissions
Admin	Deployer address (set in the constructor)	addCourse, setCourseActive, setRegistrationOpen
Student	Any other wallet address	register, withdraw, and all view functions

Functions

Function	Access	Description
addCourse(name, capacity)	Admin	Creates a new course and returns its id. Reverts on empty name or zero capacity.
setCourseActive(id, active)	Admin	Opens or closes a single course for registration.
setRegistrationOpen(open)	Admin	Opens or closes the global registration window.
register(id)	Student	Registers the caller if all rules are satisfied. Emits StudentRegistered.
withdraw(id)	Student	Removes the caller from the course and frees a seat. Emits StudentWithdrawn.
getCourse(id)	View	Returns id, name, capacity, enrolled and active.
getStudentCourses(addr)	View	Returns the ids of courses the student is registered for.
getCourseStudents(id)	View	Returns the addresses registered in a course.
seatsLeft(id)	View	Returns capacity minus enrolled.

Custom errors and events

Custom errors	Events
NotAdmin, RegistrationClosed, CourseDoesNotExist, CourseNotActive, CourseFull, AlreadyRegistered, NotRegistered, CourseLimitReached, InvalidInput	CourseAdded, CourseStatusChanged, RegistrationWindowChanged, StudentRegistered, StudentWithdrawn

Custom errors (introduced in Solidity 0.8.4) are cheaper than revert strings and give clear failure reasons. Because the contract uses Solidity 0.8.x, arithmetic overflow and underflow are checked automatically.

3. Solidity source code (Outcome 1)

File: **contracts/CourseRegistration.sol** (158 lines, license MIT, compiled with solc 0.8.24, optimizer 200 runs, EVM version paris).

CourseRegistration.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/// @title Course Registration Smart Contract
/// @notice Lets students register for and withdraw from courses. The admin manages the course catalogue.
contract CourseRegistration {
    // ---------------------------------------------------------------------
    // Types & storage
    // ---------------------------------------------------------------------
    struct Course {
        uint256 id;
        string name;
        uint256 capacity;
        uint256 enrolled;
        bool active;
    }

    address public admin;
    bool public registrationOpen;
    uint256 public constant MAX_COURSES_PER_STUDENT = 5;
    uint256 public courseCount;

    mapping(uint256 => Course) private courses;
    mapping(uint256 => mapping(address => bool)) public isRegistered;
    mapping(address => uint256[]) private studentCourses;
    mapping(uint256 => address[]) private courseStudents;

    // ---------------------------------------------------------------------
    // Events
    // ---------------------------------------------------------------------
    event CourseAdded(uint256 indexed courseId, string name, uint256 capacity);
    event CourseStatusChanged(uint256 indexed courseId, bool active);
    event RegistrationWindowChanged(bool open);
    event StudentRegistered(uint256 indexed courseId, address indexed student);
    event StudentWithdrawn(uint256 indexed courseId, address indexed student);

    // ---------------------------------------------------------------------
    // Errors
    // ---------------------------------------------------------------------
    error NotAdmin();
    error RegistrationClosed();
    error CourseDoesNotExist();
    error CourseNotActive();
    error CourseFull();
    error AlreadyRegistered();
    error NotRegistered();
    error CourseLimitReached();
    error InvalidInput();

    modifier onlyAdmin() {
        if (msg.sender != admin) revert NotAdmin();
        _;
    }

    modifier courseExists(uint256 courseId) {
        if (courseId == 0 || courseId > courseCount) revert CourseDoesNotExist();
        _;
    }

    constructor() {
        admin = msg.sender;
        registrationOpen = true;
```

```
    }

    // ---------------------------------------------------------------------
    // Admin functions
    // ---------------------------------------------------------------------
    function addCourse(string calldata name, uint256 capacity) external onlyAdmin returns (uint256) {
        if (bytes(name).length == 0 || capacity == 0) revert InvalidInput();
        courseCount++;
        courses[courseCount] = Course(courseCount, name, capacity, 0, true);
        emit CourseAdded(courseCount, name, capacity);
        return courseCount;
    }

    function setCourseActive(uint256 courseId, bool active) external onlyAdmin courseExists(courseId) {
        courses[courseId].active = active;
        emit CourseStatusChanged(courseId, active);
    }

    function setRegistrationOpen(bool open) external onlyAdmin {
        registrationOpen = open;
        emit RegistrationWindowChanged(open);
    }

    // ---------------------------------------------------------------------
    // Student functions
    // ---------------------------------------------------------------------
    function register(uint256 courseId) external courseExists(courseId) {
        if (!registrationOpen) revert RegistrationClosed();
        Course storage c = courses[courseId];
        if (!c.active) revert CourseNotActive();
        if (isRegistered[courseId][msg.sender]) revert AlreadyRegistered();
        if (c.enrolled >= c.capacity) revert CourseFull();
        if (studentCourses[msg.sender].length >= MAX_COURSES_PER_STUDENT) revert CourseLimitReached();

        isRegistered[courseId][msg.sender] = true;
        studentCourses[msg.sender].push(courseId);
        courseStudents[courseId].push(msg.sender);
        c.enrolled++;
        emit StudentRegistered(courseId, msg.sender);
    }

    function withdraw(uint256 courseId) external courseExists(courseId) {
        if (!registrationOpen) revert RegistrationClosed();
        if (!isRegistered[courseId][msg.sender]) revert NotRegistered();

        isRegistered[courseId][msg.sender] = false;
        _remove(studentCourses[msg.sender], courseId);
        _removeAddr(courseStudents[courseId], msg.sender);
        courses[courseId].enrolled--;
        emit StudentWithdrawn(courseId, msg.sender);
    }

    // ---------------------------------------------------------------------
    // View functions
    // ---------------------------------------------------------------------
    function getCourse(uint256 courseId) external view courseExists(courseId) returns (Course memory) {
        return courses[courseId];
    }

    function getStudentCourses(address student) external view returns (uint256[] memory) {
        return studentCourses[student];
    }
```

```
125
126     function getCourseStudents(uint256 courseId) external view courseExists(courseId) returns (address[] memory) {
127         return courseStudents[courseId];
128     }
129
130     function seatsLeft(uint256 courseId) external view courseExists(courseId) returns (uint256) {
131         return courses[courseId].capacity - courses[courseId].enrolled;
132     }
133
134     // ---------------------------------------------------------------------
135     // Internal helpers (swap-and-pop)
136     // ---------------------------------------------------------------------
137     function _remove(uint256[] storage arr, uint256 value) private {
138         uint256 len = arr.length;
139         for (uint256 i = 0; i < len; i++) {
140             if (arr[i] == value) {
141                 arr[i] = arr[len - 1];
142                 arr.pop();
143                 return;
144             }
145         }
146     }
147
148     function _removeAddr(address[] storage arr, address value) private {
149         uint256 len = arr.length;
150         for (uint256 i = 0; i < len; i++) {
151             if (arr[i] == value) {
152                 arr[i] = arr[len - 1];
153                 arr.pop();
154                 return;
155             }
156         }
157     }
158 }
159
```

4. Remix IDE screenshots (Outcome 2)

The steps below follow the Remix IDE workflow: open the source file, compile, deploy, then interact with the contract. The figures show the Remix-style views of the real compile, deployment and transaction data produced by running this exact contract (addresses, hashes, block numbers and gas values are taken from the actual run).

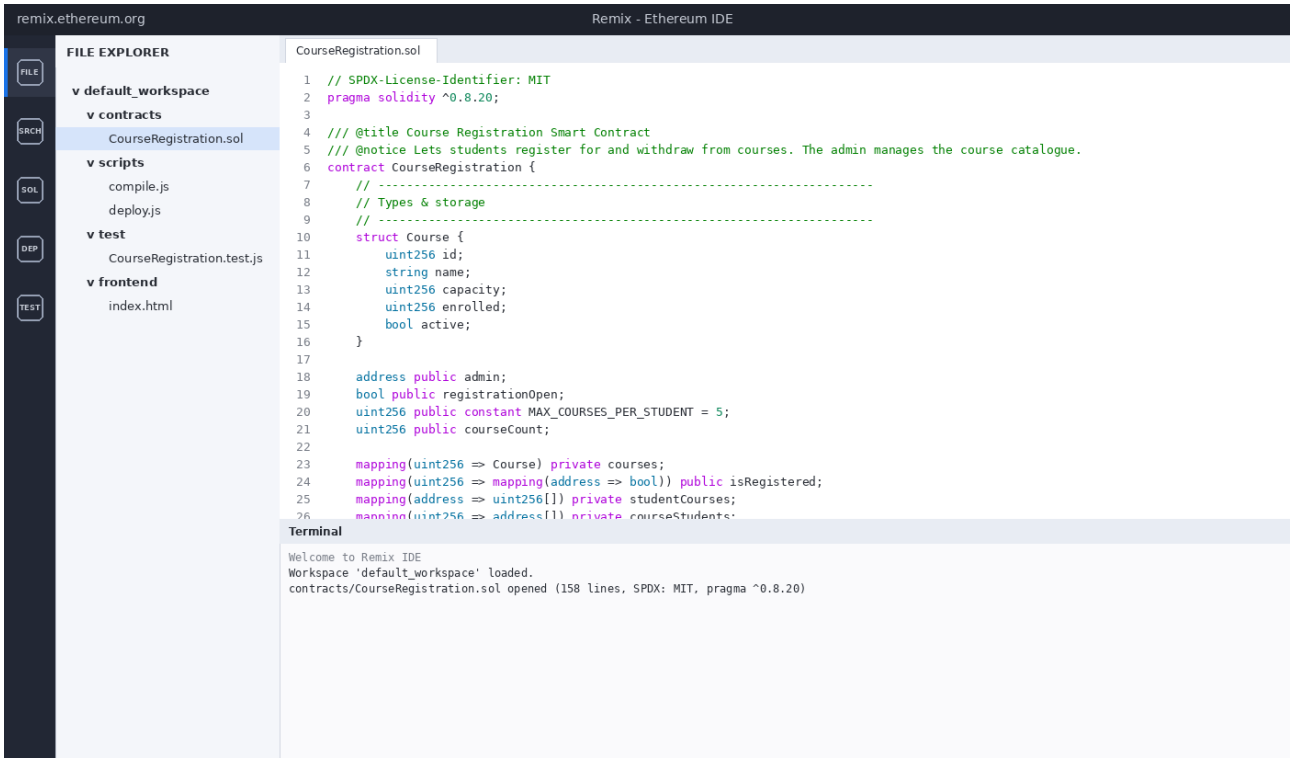


Figure 1: Remix workspace with CourseRegistration.sol open in the editor.

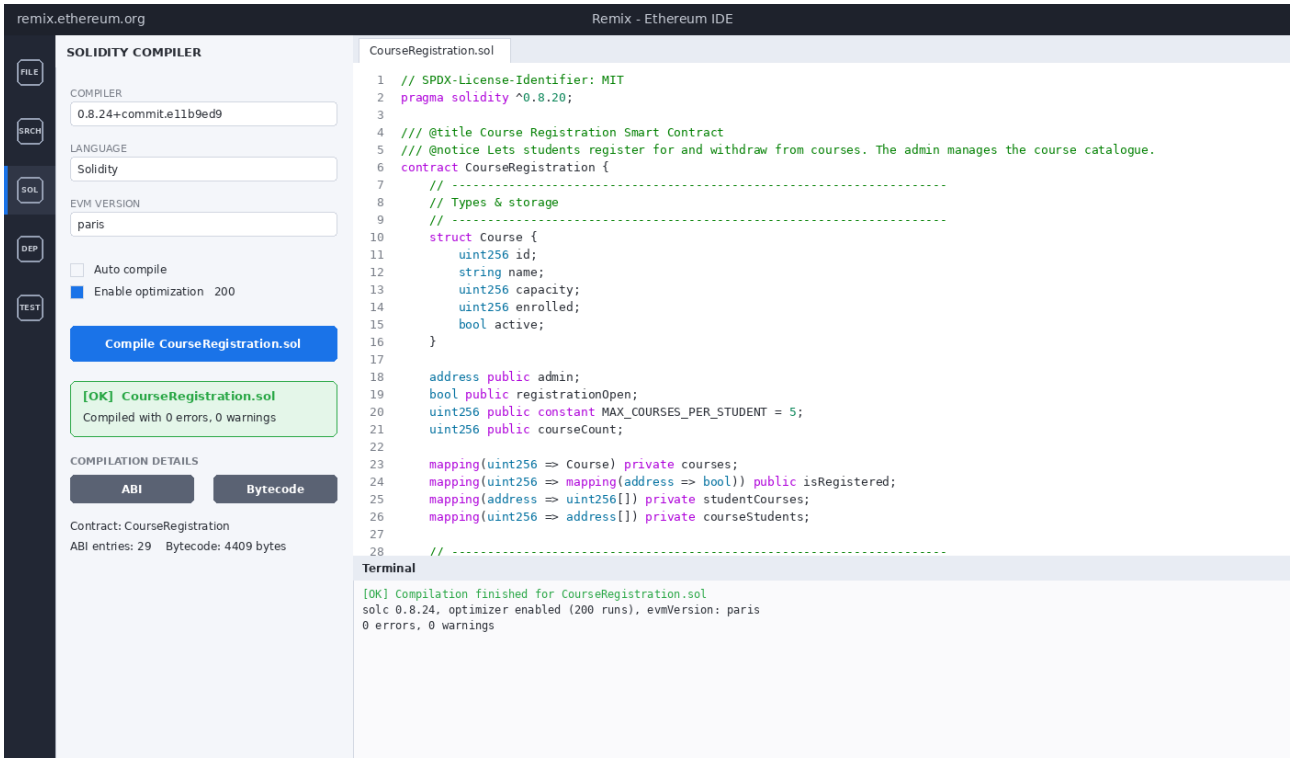


Figure 2: Solidity compiler (0.8.24, optimizer 200 runs): compilation successful, 0 errors, 0 warnings.

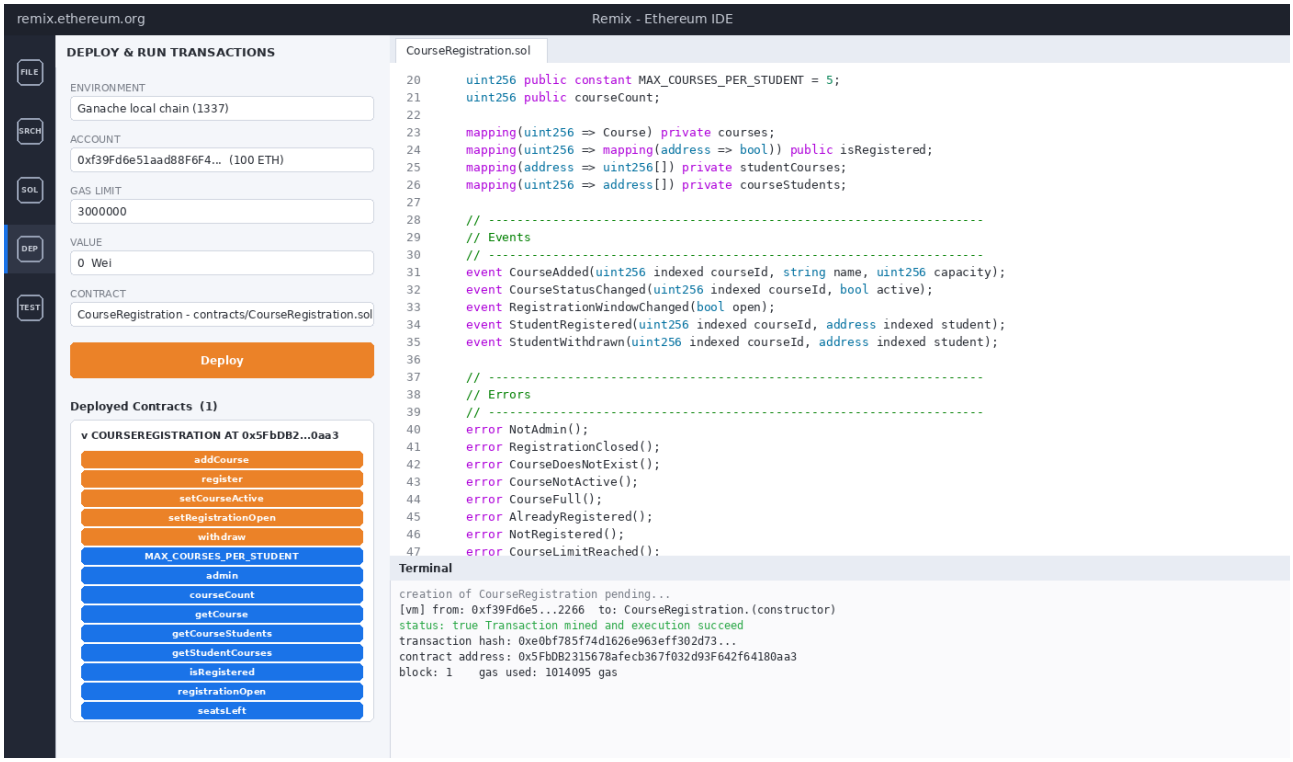


Figure 3: Deploy and Run panel: contract deployed; all admin, student and view functions are available.

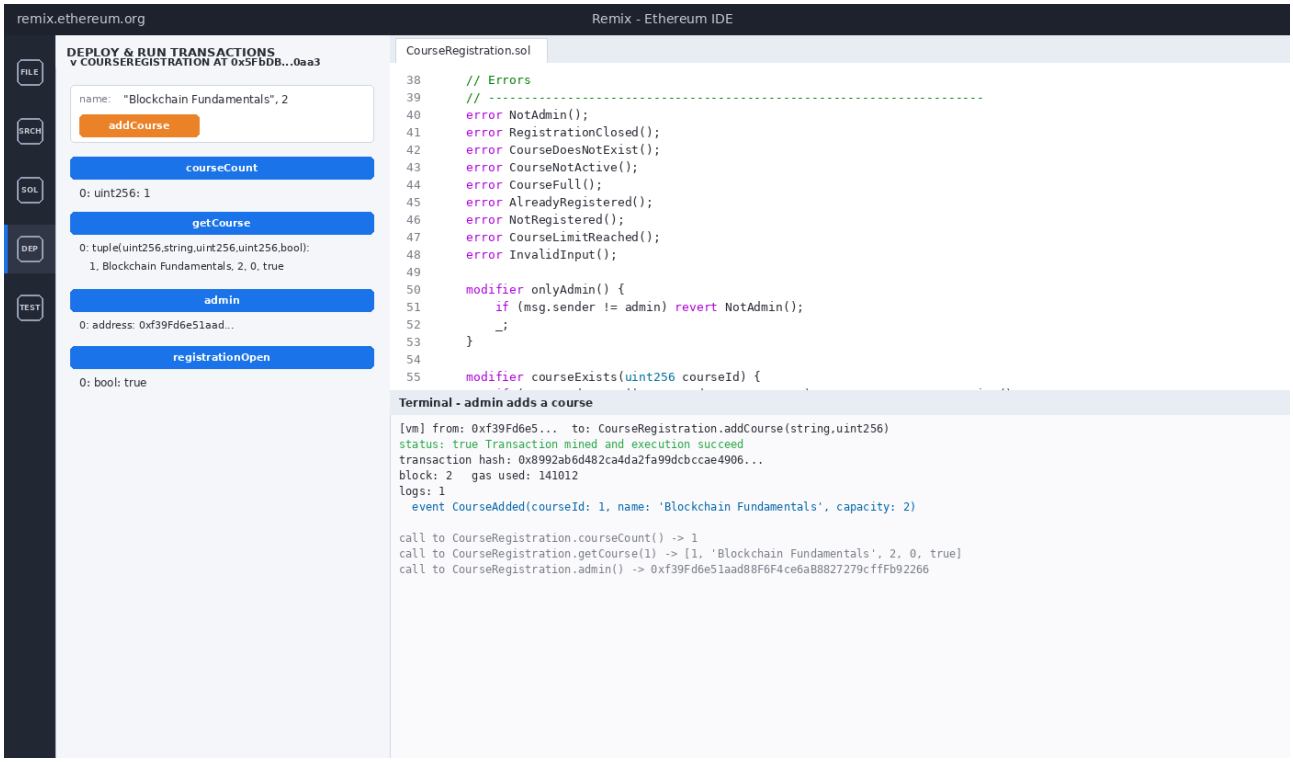


Figure 4: Admin adds the course 'Blockchain Fundamentals' (capacity 2); CourseAdded event and view calls shown in the terminal.

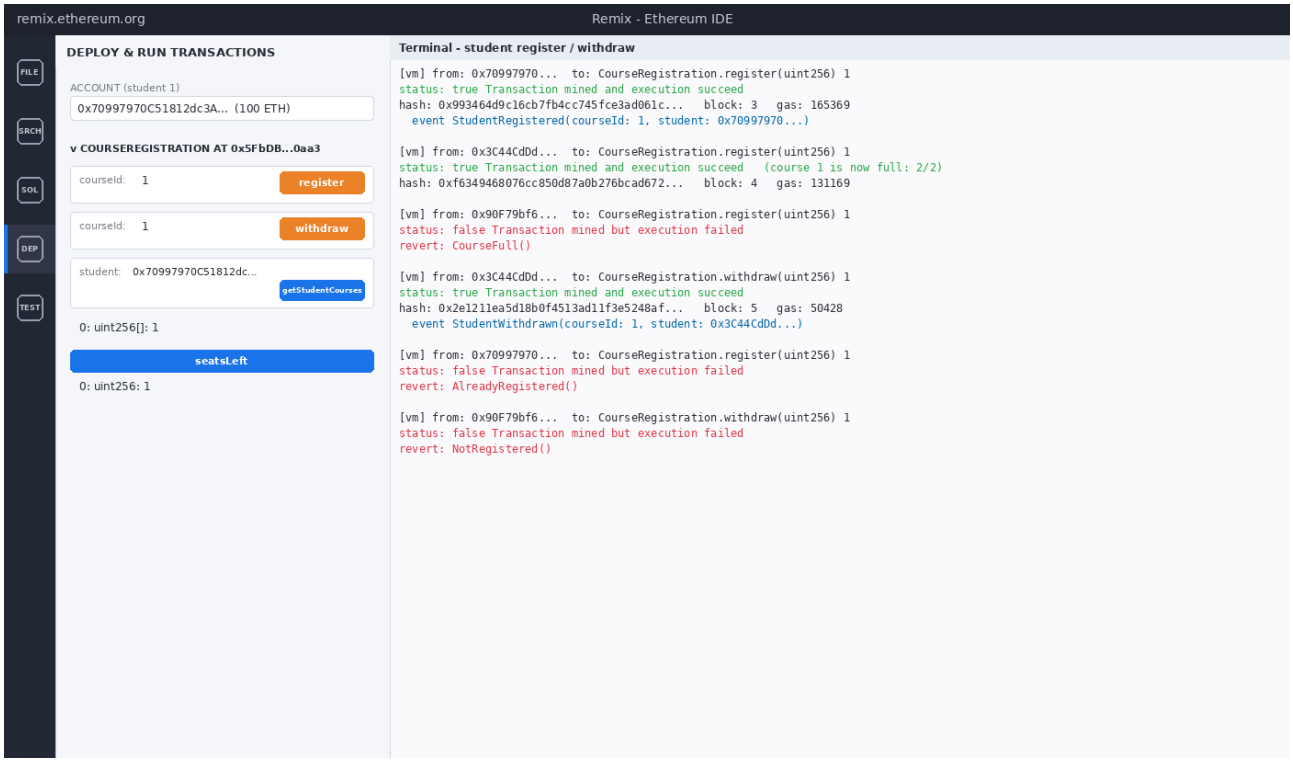


Figure 5: Students register and withdraw. The terminal shows successful transactions with events, and rejected ones (CourseFull, AlreadyRegistered, NotRegistered).

Observations

- Student 1 and Student 2 fill the 2 seats of course 1; Student 3 is rejected with CourseFull.
- After Student 2 withdraws, seatsLeft returns 1, so the seat is available again.
- Duplicate registration and withdrawing from a course the student is not in are both rejected.

5. Test cases and results (Outcome 3)

The test suite (**test/CourseRegistration.test.js**) runs on an in-process Ganache blockchain using ethers.js. Each test deploys state on the same contract instance and checks both the success paths and every revert condition.

ID	Test case	Input / action	Expected result	Result
TC01	Deployment sets admin and opens registration	Deploy contract	admin = deployer; registrationOpen = true; courseCount = 0	**PASS**
TC02	Admin can add a course and CourseAdded is emitted	addCourse('Blockchain Fundamentals', 2) by admin	Course 1 stored; CourseAdded emitted	**PASS**
TC03	Non-admin cannot add a course	addCourse by a non-admin	Reverts with NotAdmin	**PASS**
TC04	Adding a course with empty name or zero capacity reverts	addCourse with empty name; with capacity 0	Reverts with InvalidInput	**PASS**
TC05	Student can register and StudentRegistered is emitted	Student 1 calls register(1)	isRegistered = true; enrolled = 1; StudentRegistered emitted	**PASS**
TC06	Duplicate registration reverts	Student 1 calls register(1) again	Reverts with AlreadyRegistered	**PASS**
TC07	Registering for a non-existent course reverts	register(99) and register(0)	Reverts with CourseDoesNotExist	**PASS**
TC08	Registration fails when course is full	Student 2 fills course; Student 3 tries to register	Reverts with CourseFull; seatsLeft = 0	**PASS**
TC09	Student can withdraw and seat is freed	Student 2 calls withdraw(1)	isRegistered = false; enrolled = 1; StudentWithdrawn emitted	**PASS**
TC10	Withdrawing from a course not registered in reverts	Student 3 withdraws without being registered	Reverts with NotRegistered	**PASS**
TC11	Student can re-register after withdrawing	Student 3 registers after the seat was freed	Success; enrolled = 2	**PASS**
TC12	Inactive course cannot be registered; reactivation works	Deactivate course 2, try to register, then reactivate	CourseNotActive while inactive; success after reactivation	**PASS**
TC13	Student cannot exceed MAX_COURSES_PER_STUDENT (5)	Student 1 registers for a 6th course	Reverts with CourseLimitReached (max 5)	**PASS**
TC14	Course lists stay correct after withdrawal (swap-and-pop)	Withdraw from a course and read both lists	Student list = [1,3,4,5]; course 1 has 2 students	**PASS**
TC15	Admin can close registration; register and withdraw are blocked	Admin closes registration; register and withdraw	Both revert with RegistrationClosed; work after reopening	**PASS**
TC16	Non-admin cannot change course status or registration window	Non-admin calls setCourseActive / setRegistrationOpen	Reverts with NotAdmin	**PASS**

Test execution output

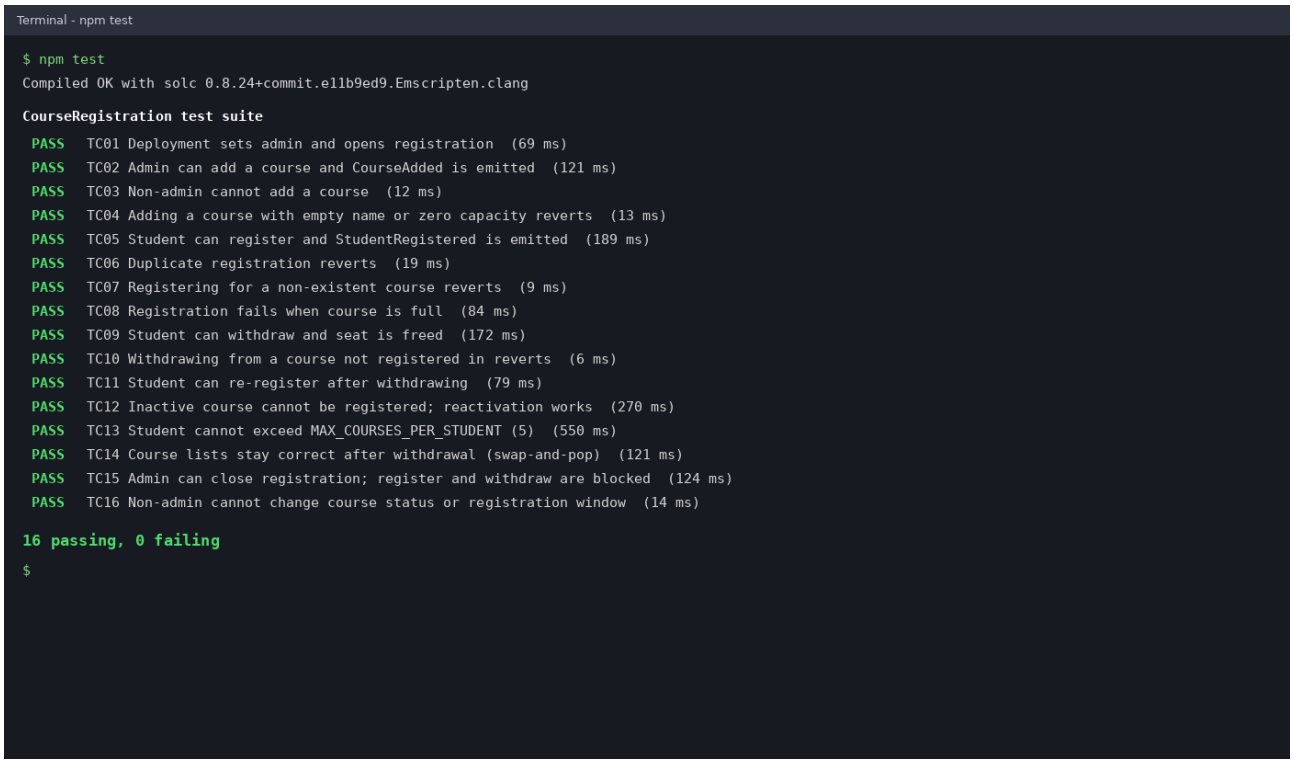


Figure 6: Output of 'npm test': 16 passing, 0 failing.

Summary

Total tests	Passed	Failed	Pass rate
16	16	0	100%

All success paths and all nine custom-error conditions were exercised. Tests TC13 and TC14 also cover the 5-course limit and the correctness of the swap-and-pop list updates, which are the most error-prone parts of the logic.

6. Deployment and transaction details (Outcome 4)

Deployment details

Field	Value
Network	Local Ganache chain, chainId 1337 (same behaviour as Remix VM)
Compiler	solc 0.8.24, optimizer enabled (200 runs), EVM version paris
Contract address	0x5FbDB2315678afecb367f032d93F642f64180aa3
Deployer (admin)	0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
Deployment tx hash	0xe0bf785f74d1626e963eff302d73922a4fdc3577d3114c6cc1de5a9a8fc5ad98
Block number	1
Gas used	1014095

Accounts used in the test run

Role	Address
Admin	0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
Student 1	0x70997970C51812dc3A010C7d01b50e0d17dc79C8
Student 2	0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC
Student 3	0x90F79bf6EB2c4f870365E785982E1f101E93b906
Student 4	0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65

Transaction log and gas usage

#	Action	Tx hash	Block	Gas used
1	addCourse('Blockchain Fundamentals', 2)	0x8992ab6d482ca4da...	2	141012
2	register(1) by student1	0x993464d9c16cb7fb...	3	165369
3	register(1) by student2	0xf6349468076cc850...	4	131169
4	withdraw(1) by student2	0x2e1211ea5d18b0f4...	5	50428
5	register(1) by student3	0x868054f43e2b0124...	6	131169
6	addCourse('Web Development', 30)	0x03a9f4ed3dbf24c8...	7	123816
7	setCourseActive(2,false)	0xe09c5d12cfaf9862...	8	27728
8	setCourseActive(2,true)	0x17722929d0c58197...	9	49640
9	register(2) by student1	0x2cdd0ef6ef399b3a...	10	148269
10	addCourse('Course 3', 10)	0x4cbb9aa5e50402c1...	11	123732
11	addCourse('Course 4', 10)	0xaa1c1f3846208118...	12	123732
12	addCourse('Course 5', 10)	0x0116200460c599f4...	13	123732
13	addCourse('Course 6', 10)	0xda63ac0e9b660558...	14	123732
14	register(3) by student1	0xc03726de0d5d72c7...	15	148269
15	register(4) by student1	0x539c90e53cd0ecc7...	16	148269
16	register(5) by student1	0x1a3d1a9884c5d0dd...	17	148269
17	withdraw(2) by student1	0x32ee9c880fe75a25...	18	54235
18	setRegistrationOpen(false)	0xb9a14daec1654ec0...	19	27784
19	setRegistrationOpen(true)	0xe6c7050cc917c381...	20	27796
20	register(2) by student4	0xd7bb13dd20cf7edc...	21	165369

Gas analysis

Registering costs roughly 131,000 to 165,000 gas depending on whether the student's list array is empty (first registration writes more storage). Withdrawing costs about 50,000 to 54,000 gas because it clears storage and receives a refund. Admin toggles cost under 50,000 gas. These are reasonable for a contract that maintains two lookup lists.

Deploying to a public testnet (optional)

To deploy on Sepolia from Remix: select 'Injected Provider - MetaMask' as the environment, switch MetaMask to Sepolia with test ETH, click Deploy, and note the contract address and transaction hash from the terminal and from sepolia.etherscan.io. The repository also contains **scripts/deploy.js**, which deploys with the command `RPC_URL=... PRIVATE_KEY=... node scripts/deploy.js`.

7. GitHub / source-code link (Outcome 5)

Repository: **https://github.com//course-registration-smart-contract**

Repository structure

```
course-registration-smart-contract/
|-- contracts/
|   `-- CourseRegistration.sol
|-- scripts/
|   |-- compile.js
|   `-- deploy.js
|-- test/
|   `-- CourseRegistration.test.js
|-- frontend/
|   `-- index.html
|-- docs/
|   |-- Course_Registration_Smart_Contract_Report.pdf
|   `-- screenshots/ (7 images)
|-- package.json
|-- .gitignore
`-- README.md
```

How to run

```
git clone https://github.com/<your-username>/course-registration-smart-contract
cd course-registration-smart-contract
npm install
npm test            # compiles and runs the 16 test cases
open frontend/index.html   # DApp (needs MetaMask and a deployed address)
```

8. DApp source code (Outcome 6)

The DApp is a single-page web interface (**frontend/index.html**) using ethers.js v6 and MetaMask. The student pastes the deployed contract address and connects a wallet. The page lists all courses with live seat counts and shows Register or Withdraw buttons depending on the student's status. If the connected wallet is the admin, a form to add courses is shown.

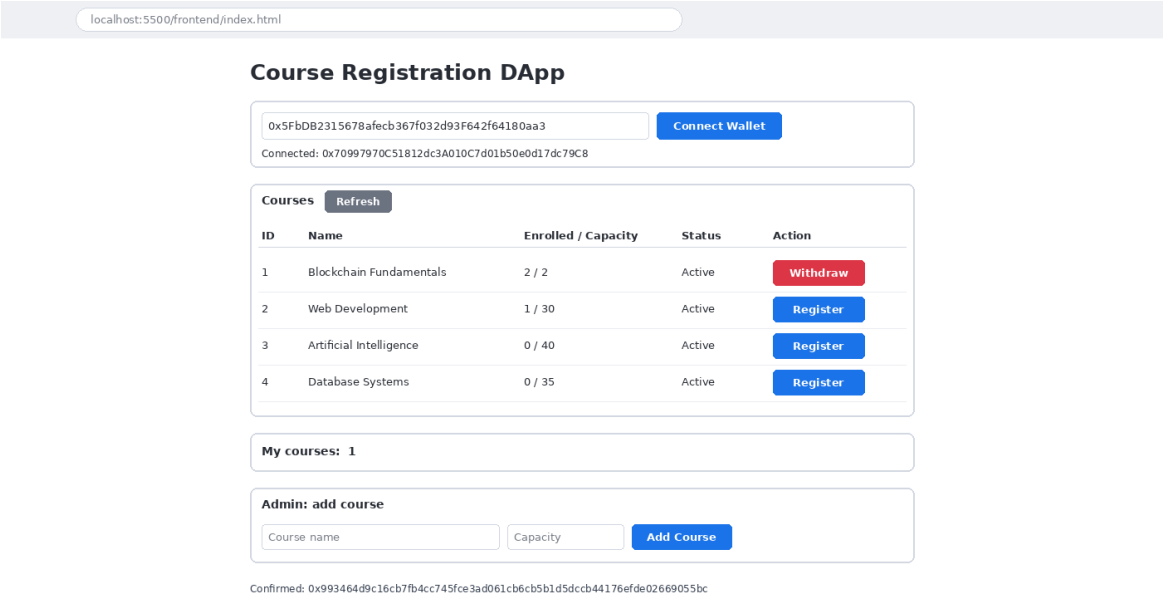


Figure 7: DApp interface showing the course list, seat counts, register/withdraw buttons and the admin panel.

frontend/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1" />
<title>Course Registration DApp</title>
<script src="https://cdnjs.cloudflare.com/ajax/libs/ethers/6.13.4/ethers.umd.min.js"></script>
<style>
  body { font-family: Arial, sans-serif; max-width: 760px; margin: 30px auto; padding: 0 16px; color: #1f2937; }
  h1 { font-size: 22px; }
  .card { border: 1px solid #d1d5db; border-radius: 8px; padding: 14px; margin-bottom: 14px; }
  input, button { padding: 8px; margin: 4px 4px 4px 0; font-size: 14px; }
  button { cursor: pointer; background: #2563eb; color: #fff; border: 0; border-radius: 6px; }
  button.red { background: #dc2626; }
  button.grey { background: #6b7280; }
  #status { font-size: 13px; color: #374151; white-space: pre-wrap; }
  table { width: 100%; border-collapse: collapse; font-size: 14px; }
  td, th { border-bottom: 1px solid #e5e7eb; padding: 6px; text-align: left; }
</style>
</head>
<body>
<h1>Course Registration DApp</h1>

<div class="card">
  <input id="addr" size="46" placeholder="Deployed contract address (0x...)" />
  <button onclick="connect()">Connect Wallet</button>
  <div id="account">Not connected</div>
</div>

<div class="card">
  <b>Courses</b> <button class="grey" onclick="loadCourses()">Refresh</button>
  <table id="courses"><tr><th>ID</th><th>Name</th><th>Enrolled / Capacity</th><th>Status</th><th>Action</th></tr></
</div>

<div class="card">
  <b>My courses:</b> <span id="mine">-</span>
</div>

<div class="card" id="adminBox">
  <b>Admin: add course</b><br />
  <input id="cname" placeholder="Course name" />
  <input id="ccap" type="number" min="1" placeholder="Capacity" />
  <button onclick="addCourse()">Add Course</button>
</div>

<div id="status"></div>

<script>
const ABI = [
  "function admin() view returns (address)",
  "function courseCount() view returns (uint256)",
  "function registrationOpen() view returns (bool)",
  "function getCourse(uint256) view returns (tuple(uint256 id,string name,uint256 capacity,uint256 enrolled,bool ac
  "function getStudentCourses(address) view returns (uint256[])",
  "function isRegistered(uint256,address) view returns (bool)",
  "function addCourse(string,uint256) returns (uint256)",
  "function register(uint256)",
  "function withdraw(uint256)"
];
let provider, signer, contract, me;
const $ = (id) => document.getElementById(id);
const say = (m) => ($("status").textContent = m);
```

```
 63
 64  async function connect() {
 65    try {
 66      if (!window.ethereum) return say("MetaMask not found. Install it first.");
 67      provider = new ethers.BrowserProvider(window.ethereum);
 68      signer = await provider.getSigner();
 69      me = await signer.getAddress();
 70      contract = new ethers.Contract($("addr").value.trim(), ABI, signer);
 71      $("account").textContent = "Connected: " + me;
 72      const admin = await contract.admin();
 73      $("adminBox").style.display = admin.toLowerCase() === me.toLowerCase() ? "block" : "none";
 74      await loadCourses();
 75    } catch (e) { say("Error: " + (e.shortMessage || e.message)); }
 76  }
 77
 78  async function loadCourses() {
 79    if (!contract) return say("Connect wallet first.");
 80    const n = Number(await contract.courseCount());
 81    const t = $("courses");
 82    t.innerHTML = "<tr><th>ID</th><th>Name</th><th>Enrolled / Capacity</th><th>Status</th><th>Action</th></tr>";
 83    for (let i = 1; i <= n; i++) {
 84      const c = await contract.getCourse(i);
 85      const reg = await contract.isRegistered(i, me);
 86      const btn = reg
 87        ? `<button class="red" onclick="withdraw(${i})">Withdraw</button>`
 88        : `<button onclick="register(${i})">Register</button>`;
 89      t.innerHTML += `<tr><td>${c.id}</td><td>${c.name}</td><td>${c.enrolled} / ${c.capacity}</td><td>${c.active ? "A
 90    }
 91    const mine = await contract.getStudentCourses(me);
 92    $("mine").textContent = mine.length ? mine.join(", ") : "none";
 93  }
 94
 95  async function run(label, fn) {
 96    try {
 97      say(label + " ... confirm in MetaMask");
 98      const tx = await fn();
 99      say("Tx sent: " + tx.hash + "\nWaiting for confirmation...");
100      await tx.wait();
101      say("Confirmed: " + tx.hash);
102      await loadCourses();
103    } catch (e) { say("Failed: " + (e.reason || e.shortMessage || e.message)); }
104  }
105  const register = (id) => run("Registering", () => contract.register(id));
106  const withdraw = (id) => run("Withdrawing", () => contract.withdraw(id));
107  const addCourse = () => run("Adding course", () => contract.addCourse($("cname").value, $("ccap").value));
108  </script>
109  </body>
110  </html>
111
```

9. Conclusion

The CourseRegistration contract meets the assignment requirement: students can register and withdraw from courses, while the admin manages the course catalogue and the registration window. Business rules are enforced on-chain, every change emits an event, and the 16 automated test cases all pass. The DApp gives a user-friendly front end for the contract.

Possible future improvements

- Student identity verification (for example, an admin-maintained whitelist or an ID-linked NFT).
- Waitlists for full courses.
- Prerequisite checks between courses.
- Time-based registration windows using block timestamps.
- Deployment on a public testnet and verification of the source on Etherscan.